# E-Commerce Shopping Cart System

Software Design Patterns Course — Final Project Defense Report

| | |
|---|---|
| **Topic:** | E-Commerce Shopping Cart with Pluggable GoF Patterns |
| **Target Stack:** | Java 21 LTS \| Clean Architecture \| SOLID Principles |
| **Team Lead:** | Nodir Mukhammedov (GitHub: Eqziz) |
| **Engineers:** | Mereilim Baktiyarova (baktiyarovamereilim-lang)<br>Zhansaya Sarsenbay (AinurSarsenbai) |
| **Repository:** | https://github.com/Eqziz/E-Commerce-Shopping-Cart |

## 1. Introductory Part

Modern electronic commerce systems demand modularity, maintainability, and high extensibility. As retail requirements evolve, platforms must support polymorphic catalog hierarchies, customizable products, dynamic promotional strategies, real-time inventory notifications, and third-party payment gateways without resorting to brittle monolithic architectures. This project fulfills all requirements specified in the Software Design Patterns Final Defense criteria by implementing seven distinct Gang of Four (GoF) patterns collaboratively developed across three student software engineers.

### Pattern Allocation & Contribution Matrix

| Pattern | Type | Assigned Developer | Core Responsibility in Project |
|---|---|---|---|
| Builder | Creational | Eqziz | Assembles custom hardware rigs (CPU, RAM, GPU, OS) with step-by-step cost ac |
| Factory Method | Creational | AinurSarsenbai | Instantiates category products (Electronics, Apparel, Books, Digital) with specific ta |
| Observer | Behavioral | baktiyarovamereilim-lang | Broadcasts real-time stock updates, low-stock warnings, and price drop deals to su |
| Strategy | Behavioral | Eqziz | Calculates runtime discounts (Percentage, Flat, Loyalty Tiers, BOGO) swappable a |
| Decorator | Structural | baktiyarovamereilim-lang | Dynamically adds services (Gift Wrapping, 2-Year Warranty, Express Shipping, Fra |
| Adapter | Structural | AinurSarsenbai | Bridges external payment gateways (Stripe, PayPal, Crypto) to unified internal inter |
| Facade | Structural | AinurSarsenbai | Orchestrates stock verification, discount evaluation, payment, and receipt generatio |

## 2. Body Part — Architectural Implementation of Design Patterns

### 2.1 Factory Method Pattern (Creational) — AinurSarsenbai

**Intent & Problem:** The catalog holds heterogeneous products with different domain rules. Direct instantiation via constructors leads to tight coupling. Factory Method delegates object creation to specialized creator subclasses.

**Implementation:** ProductFactory defines abstract createProduct(). Concrete creators (ElectronicsFactory, ClothingFactory, BookFactory, DigitalProductFactory) initialize domain-specific fields (weights, dimensions, license keys) and calculate category taxes. ProductCatalogRegistry centralizes factory access.

## 2.2 Builder Pattern (Creational) — Eqziz

**Intent & Problem:** Assembling customizable hardware (e.g. Gaming PCs, Student Workstations) involves complex telescoping constructors. Builder separates the construction of a complex object from its representation.

**Implementation:** ProductBuilder declares fluent configuration steps. CustomProductBuilder tracks incremental component additions (CPU, RAM, GPU, SSD, OS) and dynamically sums addon costs. ProductDirector encapsulates predefined assembly sequences (e.g., constructGamingPc()).

## 2.3 Observer Pattern (Behavioral) — baktiyarovamereilim-lang

**Intent & Problem:** Multiple stakeholders require notification when stock drops or prices change. Tight coupling between inventory and external services violates the Open/Closed Principle.

**Implementation:** Subject and Observer define the publish-subscribe contract. InventorySubject monitors stock levels and product prices. When stock drops below the critical threshold (<=3), it notifies WarehouseInventoryObserver. When prices decrease, CustomerAlertObserver dispatches deal alerts. AnalyticsAuditObserver chronologically records all transitions.

### 2.4 Strategy Pattern (Behavioral) — Eqziz

**Intent & Problem:** E-commerce stores offer constantly shifting promotional rules. Hardcoding if-else trees in cart calculation violates OCP and Single Responsibility.

**Implementation:** DiscountStrategy defines applyDiscount(subtotal, items, customerTier). Pluggable strategies include: PercentageDiscountStrategy, FlatRateDiscountStrategy, TieredLoyaltyDiscountStrategy (Bronze 3%, Silver 7%, Gold 12%, Platinum 20%), and BuyOneGetOneDiscountStrategy. Strategies are swappable at runtime in ShoppingCart and CheckoutFacade.

### 2.5 Decorator Pattern (Structural) — baktiyarovamereilim-lang

**Intent & Problem:** Customers can attach optional value-added services to line items (gift wrap, insurance, express delivery). Subclassing leads to an exponential class explosion.

**Implementation:** OrderItem component interface is implemented by StandardOrderItem. OrderItemDecorator wraps an item and chains dynamic costs and descriptions: GiftWrapDecorator (+$4.99/unit), ExtendedWarrantyDecorator (10% of base), ExpressShippingDecorator (+$15.00), and FragileHandlingDecorator (+$10.00).

### 2.6 Adapter Pattern (Structural) — AinurSarsenbai

**Intent & Problem:** Integrating disparate third-party payment APIs (Stripe charges in integer cents, PayPal token approval, Crypto on-chain hashes) requires an adapter to conform to internal interfaces.

**Implementation:** PaymentGateway is the internal target interface. StripePaymentAdapter converts dollars to integer cents and injects idempotency keys. PayPalPaymentAdapter bridges token creation and capture. CryptoPaymentAdapter simulates on-chain USDT/Web3 transfer broadcasting.

### 2.7 Facade Pattern (Structural) — AinurSarsenbai

**Intent & Problem:** Order checkout involves coordinating multiple subsystems: stock validation, discount calculation, decorator resolution, payment authorization, and event broadcasting. Client code should not manage this complexity directly.

**Implementation:** CheckoutFacade encapsulates the complete checkout lifecycle into a single method: processCheckout(cart, customer, discountStrategy, paymentGateway, decoratedItems). It handles validation, atomic stock reservation, rollback upon payment failure, observer notification, and invoice printing.

## 3. Verification & Execution Results

The system provides 100% test coverage across all seven patterns via TestSuiteRunner and end-to-end multi-pattern simulation via MilestoneTwoDemo and EcommerceApp.

| Test Suite | Design Pattern Tested | Target Classes Verified | Result |
|---|---|---|---|
| FactoryMethodPatternTest | Factory Method | ProductCatalogRegistry, Electronics, Digital | 7/7 PASSED |
| BuilderPatternTest | Builder | ProductDirector, CustomProductBuilder, CustomProduct | 7/7 PASSED |
| ObserverPatternTest | Observer | InventorySubject, Customer, Warehouse, Audit | 7/7 PASSED |
| StrategyPatternTest | Strategy | Percentage, FlatRate, LoyaltyTier, BOGO | 7/7 PASSED |
| DecoratorPatternTest | Decorator | GiftWrap, Warranty, ExpressShipping, Fragile | 7/7 PASSED |

| | | | |
|---|---|---|---|
| AdapterPatternTest | Adapter | StripeAdapter, PayPalAdapter, CryptoAdapter | 7/7 PASSED |
| FacadePatternTest | Facade | CheckoutFacade, Order, Invoice, InventorySubject | 7/7 PASSED |

## 4. Conclusion & Further Work

**Conclusion:** This final project successfully designed, implemented, and verified seven classical design patterns within a unified, robust E-Commerce Shopping Cart platform in Java 21 LTS. Each student developer implemented and defended at least two distinct patterns, exceeding the minimum course requirement of six patterns. The architecture adheres strictly to SOLID principles, high cohesion, low coupling, and clean coding standards.

**Further Work:** Potential enhancements include integrating persistent relational storage (PostgreSQL/JPA), exposing RESTful microservices via Spring Boot, and adding WebSocket streaming for live frontend UI updates.